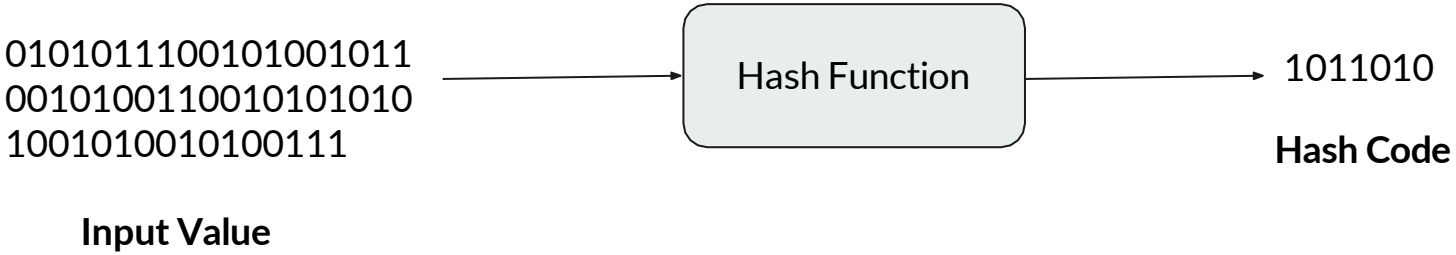
Module 4

Secure Hash Algorithm (SHA)

What is Hashing?



- A hash function is any function that can be used to map data of arbitrary size to fixed-size values, though there are some hash functions that support variable length output.
- The values returned by a hash function are called hash values, hash codes, digests, or simply hashes.

Use of hashing

1. Data Verification
2. Password Storage
3. Digital Signatures
4. Message Authentication Codes

Example of SHA-512

SHA512

This SHA512 online tool helps you calculate hash from string or binary. You can input UTF-8, UTF-16, Hex to SHA512. It also supports HMAC.

Input Type UTF-8

```
A hash function is any function that can be used to map data of arbitrary size to fixed-size
values, though there are some hash functions that support variable length output. The values
returned by a hash function are called hash values, hash codes, digests, or simply hashes.
```

☐ Remember Input

☐ Enable HMAC

Hash ☑ Auto Update

```
99b67cb618734a45d97e4cb88a10ed21421ba2950ccda8f98a2c0610f65354064139143a2904eaa882fb9341cbcaa
0890c4e5f369b80887e2a04802e4cbe0d7e
```

SHA-512

1. Overall Structure Diagram + 3 steps
2. F Function Explanation in Detail Diagram
3. Round Function Diagram
4. Extraction of 64 bit from 1024 bit Diagram

SHA-512

⊗ i/p len max 2^{128} bits
o/p – 512 bits

1) I/P block by block → 1024 bit
2) Length of msg before padding ⇒ 128 bit

8 bit
66 bit + 2

3) Last block of msg = 1024
m = 896 bits
L ⇒ 128 bit

4) M_{last} < 896 bits ⇒ 100000...

H_0 ⇒ 512 bit
8 × 64 bit
a b c d e f g h

1st 8 prime no → 89 rt
5 ⇒ 2.34134568...
64-bit

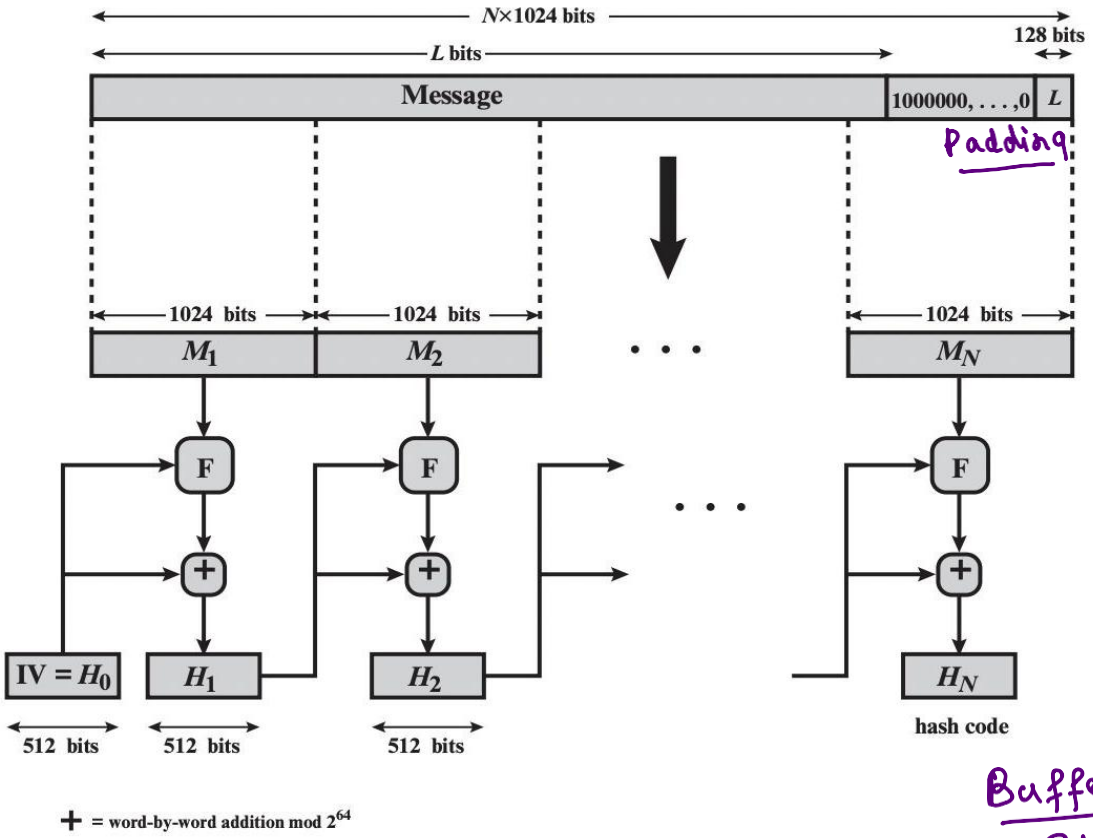


Figure 11.9 Message Digest Generation Using SHA-512

Buffers
512 bits
O/P → H_N

SHA-512 Logic

The algorithm takes as input a message with a maximum length of less than 2^{128} bits and produces as output a 512-bit message digest. The input is processed in 1024-bit blocks. Figure 11.9 depicts the overall processing of a message to produce a digest.

Step 1 **Append padding bits.** The message is padded so that its length is congruent to 896 modulo 1024 [length $\equiv$ 896(mod 1024)]. Padding is always added, even if the message is already of the desired length. Thus, the number of padding bits is in the range of 1 to 1024. The padding consists of a single 1 bit followed by the necessary number of 0 bits.

Step 2 **Append length.** A block of 128 bits is appended to the message. This block is treated as an unsigned 128-bit integer (most significant byte first) and contains the length of the original message in bits (before the padding).

The outcome of the first two steps yields a message that is an integer multiple of 1024 bits in length. In Figure 11.9, the expanded message is represented as the sequence of 1024-bit blocks $M_1, M_2, \ldots, M_N$, so that the total length of the expanded message is $N \times 1024$ bits.

Step 3 **Initialize hash buffer.** A 512-bit buffer is used to hold intermediate and final results of the hash function. The buffer can be represented as eight 64-bit registers (a, b, c, d, e, f, g, h). These registers are initialized to the following 64-bit integers (hexadecimal values):

```
a = 6A09E667F3BCC908    e = 510E527FADE682D1
b = BB67AE8584CAA73B    f = 9B05688C2B3E6C1F
c = 3C6EF372FE94F82B    g = 1F83D9ABFB41BD6B
d = A54FF53A5F1D36F1    h = 5BE0CD19137E2179
```

These values are stored in **big-endian** format, which is the most significant byte of a word in the low-address (leftmost) byte position. These words were obtained by taking the first sixty-four bits of the fractional parts of the square roots of the first eight prime numbers.

Step 4

f

80 rounds

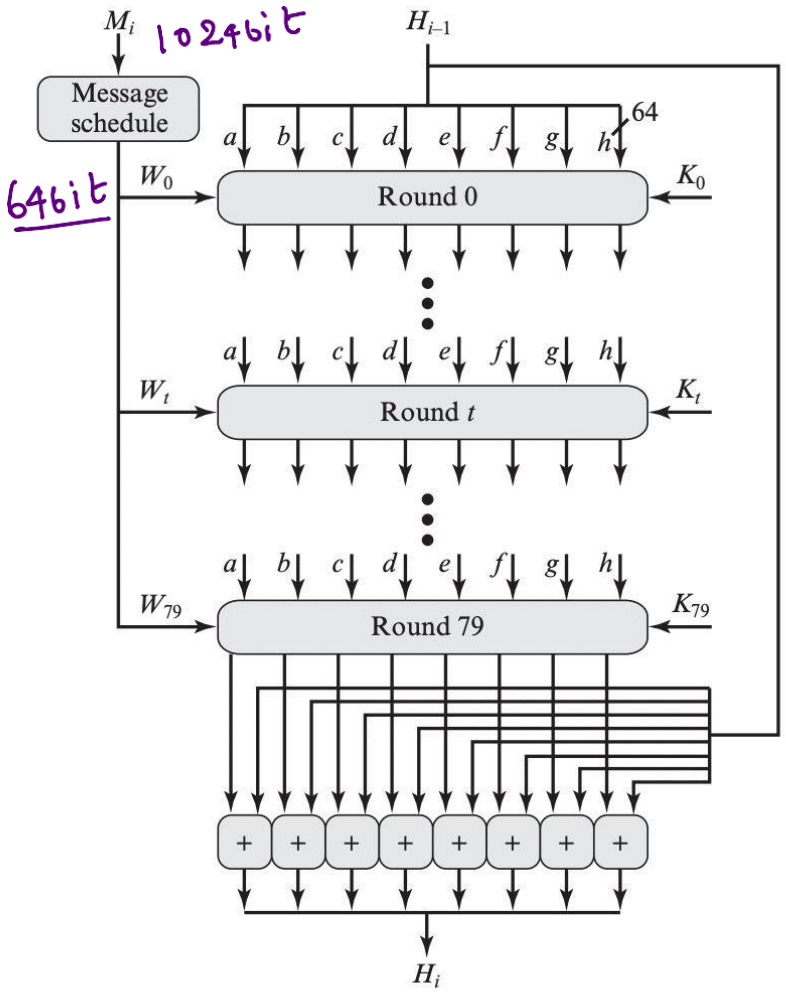


Figure 11.10 SHA-512 Processing of a Single 1024-Bit Block

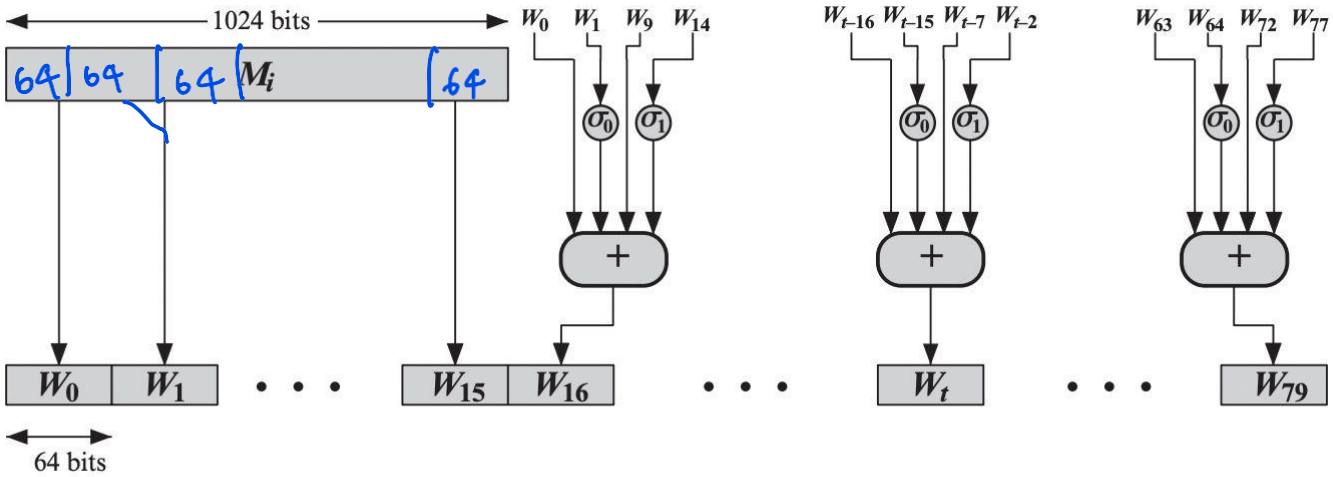
Step 4 Process message in 1024-bit (128-byte) blocks. The heart of the algorithm is a module that consists of 80 rounds; this module is labeled F in Figure 11.9. The logic is illustrated in Figure 11.10.

Each round takes as input the 512-bit buffer value, abcdefgh, and updates the contents of the buffer. At input to the first round, the buffer has the value of the intermediate hash value, H_{i-1}. Each round t makes use of a 64-bit value W_t, derived from the current 1024-bit block being processed (M_i). These values are derived using a message schedule described subsequently. Each round also makes use of an additive constant K_t, where $0 \leq t \leq 79$ indicates one of the 80 rounds. These words represent the first 64 bits of the fractional parts of the cube roots of the first 80 prime numbers. The constants provide a "randomized" set of 64-bit patterns, which should eliminate any regularities in the input data. Table 11.4 shows these constants in hexadecimal format (from left to right).

The output of the eightieth round is added to the input to the first round (H_{i-1}) to produce H_i. The addition is done independently for each of the eight words in the buffer with each of the corresponding words in H_{i-1}, using addition modulo 2^{64}.

How W_t of 64 bits translated from 1024 bit input block?

$W_0 - W_{15}$ – copied M_i



1024
0–63 W_0
63 W_1
W_2

Figure 11.12 Creation of 80-word Input Sequence for SHA-512 Processing of Single Block

$$W_t = \sigma_1^{512}(W_{t-2}) + W_{t-7} + \sigma_0^{512}(W_{t-15}) + W_{t-16}$$

$$W_{16} = \sigma_1^{512}(W_{14}) + W_9 + \sigma_0^{512}(W_1) + W_0$$

where

$$\sigma_0^{512}(x) = \mathrm{ROTR}^1(x) \oplus \mathrm{ROTR}^8(x) \oplus \mathrm{SHR}^7(x)$$
$$\sigma_1^{512}(x) = \mathrm{ROTR}^{19}(x) \oplus \mathrm{ROTR}^{61}(x) \oplus \mathrm{SHR}^6(x)$$

$\mathrm{ROTR}^n(x)$ = circular right shift (rotation) of the 64-bit argument x by n bits

$\mathrm{SHR}^n(x)$ = right shift of the 64-bit argument x by n bits with padding by zeros on the left

$+$ = addition modulo 2^{64}

SHA-512 Round Function

Let us look in more detail at the logic in each of the 80 steps of the processing of one 512-bit block (Figure 11.11). Each round is defined by the following set of equations:

$$T_1 = h + \mathrm{Ch}(e, f, g) + (\textstyle\sum_1^{512} e) + W_t + K_t$$
$$T_2 = (\textstyle\sum_0^{512} a) + \mathrm{Maj}(a, b, c)$$
$$h = g$$
$$g = f$$
$$f = e$$
$$e = d + T_1$$
$$d = c$$
$$c = b$$
$$b = a$$
$$a = T_1 + T_2$$

where

t = step number; $0 \le t \le 79$

$\mathrm{Ch}(e, f, g)$ = $(e \text{ AND } f) \oplus (\text{NOT } e \text{ AND } g)$
the conditional function: If e then f else g

$\mathrm{Maj}(a, b, c)$ = $(a \text{ AND } b) \oplus (a \text{ AND } c) \oplus (b \text{ AND } c)$
the function is true only of the majority (two or three) of the arguments are true

$(\Sigma_0^{512} a)$ = $\mathrm{ROTR}^{28}(a) \oplus \mathrm{ROTR}^{34}(a) \oplus \mathrm{ROTR}^{39}(a)$

$(\Sigma_1^{512} e)$ = $\mathrm{ROTR}^{14}(e) \oplus \mathrm{ROTR}^{18}(e) \oplus \mathrm{ROTR}^{41}(e)$

$\mathrm{ROTR}^n(x)$ = circular right shift (rotation) of the 64-bit argument x by n bits

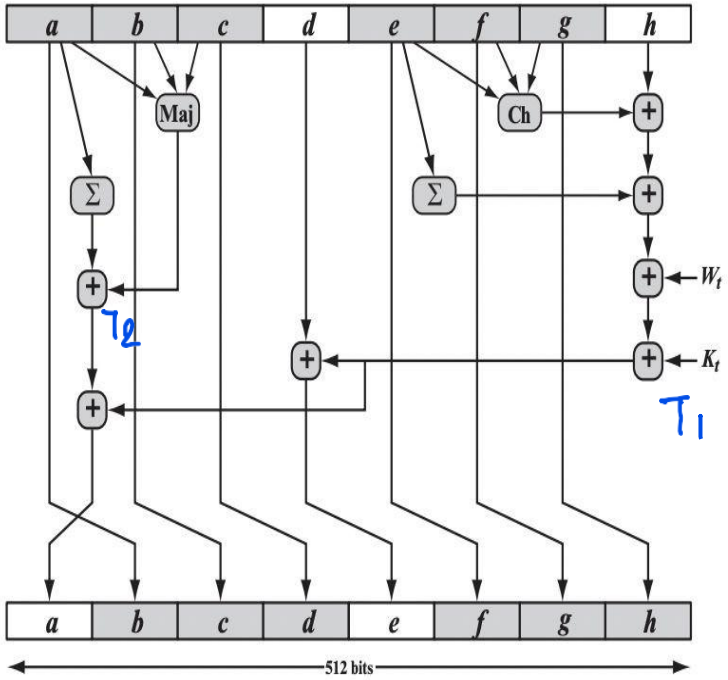


Figure 11.11 Elementary SHA-512 Operation (single round)

SHA-512 Round Function

Let us look in more detail at the logic in each of the 80 steps of the processing of one 512-bit block (Figure 11.11). Each round is defined by the following set of equations:

$$T_1 = h + \mathrm{Ch}(e, f, g) + (\textstyle\sum_1^{512} e) + W_t + K_t$$

$$T_2 = (\textstyle\sum_0^{512} a) + \mathrm{Maj}(a, b, c)$$

$$h = g$$

$$g = f$$

$$f = e$$

$$e = d + T_1$$

$$d = c$$

$$c = b$$

$$b = a$$

$$a = T_1 + T_2$$

where

t	=	step number; $0 \le t \le 79$
$\mathrm{Ch}(e, f, g)$	=	$(e \text{ AND } f) \oplus (\text{NOT } e \text{ AND } g)$ *the conditional function: If e then f else g*
$\mathrm{Maj}(a, b, c)$	=	$(a \text{ AND } b) \oplus (a \text{ AND } c) \oplus (b \text{ AND } c)$ *the function is true only of the majority (two or three) of the arguments are true*
$(\Sigma_0^{512} a)$	=	$\mathrm{ROTR}^{28}(a) \oplus \mathrm{ROTR}^{34}(a) \oplus \mathrm{ROTR}^{39}(a)$
$(\Sigma_1^{512} e)$	=	$\mathrm{ROTR}^{14}(e) \oplus \mathrm{ROTR}^{18}(e) \oplus \mathrm{ROTR}^{41}(e)$
$\mathrm{ROTR}^n(x)$	=	circular right shift (rotation) of the 64-bit argument x by n bits

$a \quad b \quad c \quad d \quad e \quad f \quad g \quad h$

$h = g$

$g = f$

$f = e$

$d = c$

$c = b$

$b = a$

$e = d + T_1$

$a = T_1 + T_2$

W_t = a 64-bit word derived from the current 1024-bit input block

K_t = a 64-bit additive constant

$+$ = addition modulo 2^{64}

Key Points about Round Function

Two observations can be made about the round function.

1. Six of the eight words of the output of the round function involve simply permutation (b, c, d, f, g, h) by means of rotation. This is indicated by shading in Figure 11.11.
2. Only two of the output words (a, e) are generated by substitution. Word e is a function of input variables (d, e, f, g, h), as well as the round word W_t and the constant K_t. Word a is a function of all of the input variables except d, as well as the round word W_t and the constant K_t.

Step 5

Step 5 Output. After all N 1024-bit blocks have been processed, the output from the Nth stage is the 512-bit message digest.

We can summarize the behavior of SHA-512 as follows:

$$H_0 = \text{IV}$$
$$H_i = \text{SUM}_{64}(H_{i-1}, \text{abcdefgh}_i)$$
$$MD = H_N$$

where

IV	=	initial value of the abcdefgh buffer, defined in step 3
abcdefgh_i	=	the output of the last round of processing of the ith message block
N	=	the number of blocks in the message (including padding and length fields)
SUM_{64}	=	addition modulo 2^{64} performed separately on each word of the pair of inputs
MD	=	final message digest value

Practice Questions:

Mr. Mathur is developing a secure application and wants to understand how the SHA-512 hashing algorithm works. To help him grasp the concept, let's consider a specific example. Mathur has a message "Hello World" that he wants to hash using SHA-512. He has read about the algorithm and knows that SHA-512 involves multiple rounds of processing, but he is not clear on what happens in a single round. Explain to Mathur the steps involved in a single round of the SHA-512 hashing algorithm when processing the message "Hello World".

Determine the value of the padding field in SHA-512 if the length of the message is

a. 1743 bits b. 2490 bits

a. Rusty wants to hash a message “secret” using the SHA-512 algorithm. Help Rusty to make the message as an exact multiple of required block size. Represent the preprocessed message made as the exact multiple of required block size in hexadecimal. [Note: ASCII value of a-97, b-98,… represented in 8 bits]

b. Consider a message of size 1024 bits to be processed using SHA512 bits. Does the SHA512 algorithm require any preprocessing of message? If so, what will be size of the message once the preprocessing is done?